

Daniil Raikhman

Roblox Developer / Roblox Game Designer / Systems Engineer

Roblox Studio • Luau --!strict • React-Lua • Zap • ProfileStore • Rojo • Wally • GitHub Actions • FTUE • Retention • Monetization

About

Roblox game developer with a strong focus on architecture, systems design, and a production-minded approach to development. I combine engineering thinking with a product/game design mindset: I design and implement game systems so they are not only functional, but also scalable, testable, secure, and maintainable.

I work confidently with Luau --!strict, build server-authoritative mechanics, typed client-server networking, persistence, UI architecture, and a quality control pipeline. I treat a project not only as a set of systems, but as a full game product: from idea, GDD, core loop, and economy to FTUE, retention, mobile UX, and Discover/RFY specifics.

I use a modern Roblox development stack: React-Lua, Flipper, Zap, ProfileStore, Rojo, Wally, Rokit, Lune, GitHub Actions. I actively use AI tools to accelerate architecture design, system scaffolding, refactoring, documentation, and tests, while keeping full control over architectural decisions, code review, and final quality.

Key Skills

Roblox Studio	Luau --!strict	Server-authoritative gameplay	Client/server architecture
React-Lua / React-Roblox	Flipper	Zap	ProfileStore
Rojo / Wally / Rokit	Signal / Promise / Trove	Lune tests	StyLua / Selene
Git / GitHub Actions / CI/CD	Code architecture	FTUE / onboarding	Retention D1/D7/D30
Game economy / progression	Monetization	Mobile UX	GDD / MVP / PROVE / V1

Experience

Roblox Developer / Systems Engineer

Independent development / personal projects • [Month Year] — Present

- Design Roblox project architecture with a clear shared / server / client split, isolating domain logic, UI, networking, and data handling.
- Implement server-authoritative systems where economy, progression, inventory, upgrades, and key gameplay actions are validated on the server.
- Extract business logic into pure modules so formulas, balance, and game rules can be tested separately from the Roblox API and Studio environment.
- Build a typed networking layer with Zap and develop UI in React-Lua with predictable state and Flipper-based animations.
- Set up player data persistence and lifecycle with ProfileStore, including profile loading, session locking, and safe data handling.
- Introduce engineering quality practices: StyLua, Selene, Lune unit tests, Rojo builds, and CI checks via GitHub Actions.
- Also work at the game/product design level: niche analysis, product gaps, GDD, core loop, economy, retention layers, social mechanics, and mobile UX.
- Use AI tools to speed up architecture design, scaffolding, refactoring, tests, and documentation without losing engineering quality.

Projects

Deep Digger

Game link: <https://www.roblox.com/games/77149464720360/>

A Roblox project focused on core loop, retention logic, marketing positioning, and a product-driven development approach. It served as a practical foundation for refining design, economy, retention, and Roblox packaging methods.

Roblox Starter / Internal Roblox Framework

Tech: Luau, React-Lua, Flipper, Zap, ProfileStore, Rojo, Wally, Lune, GitHub Actions

- Built an engineering foundation for Roblox games focused on fast MVP launch and safe project scaling.
- Designed a layered architecture: shared for logic and types, server for validation and data, client for UI, visuals, and controllers.
- Implemented a gameplay systems template using the Logic -> Manager -> Network -> UI pattern and typed HUD / server action sync.
- Prepared infrastructure for testable pure logic, CI, and shared code quality standards, plus a performance approach based on quality tiers and budgets.

Strengths

- I design systems before implementation instead of fixing architectural problems after the fact.
- I think at the product, architecture, and production-process level — not just in terms of individual scripts.
- I clearly understand client/server boundaries, progression security, and state synchronization.
- I quickly turn ideas into working prototypes and production-ready implementations.
- I use AI as a tool to accelerate development without compromising engineering quality.